

Lab exercises

Part 04 —  Large Language Models · 04-llm/lab/EXERCISES.md

Lab — Large Language Models

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. No API keys are required — everything runs offline against the code in `../code/` or against your own reasoning on paper.

1. Feel the sampling knobs (45 min)

Run `python code/sampling_strategies.py`. It trains a character-level bigram model and samples from it at different settings.

- **1a.** Set `temperature = 0.01` and `temperature = 2.0`. Which output is more "creative", and which is more often wrong? One sentence each.
- **1b.** Set `top_p = 0.1` at `temperature = 1.5`, then `top_p = 0.999` at `temperature = 0.2`. Which knob dominated the output style? Explain in one sentence why temperature flattening and nucleus truncation are *not* redundant.
- **1c.** Add a `min_p` parameter (keep only tokens with `p_i >= min_p * p_max`). Find the smallest value that eliminates the gibberish words at `temperature = 1.5`.

Deliverable: three sentences and a working `min_p` implementation.

2. Cost model under interrogation (1h)

Open `code/llm_cost_calculator.py`. Compute, on paper first, then verify with the script:

- **2a.** A support-bot request: 2,000-token system prompt (cached), 800 tokens of user context, 300 output tokens. Monthly cost at 50,000 requests with and without caching. What is the caching saving as a percentage?
- **2b.** An agent that runs 12 turns, re-sending the full transcript each turn. Plot (or tabulate) cost vs. turn count. Is growth linear or quadratic in turns, and why?
- **2c.** Same traffic, self-hosted: use the calculator's GPU memory model. Does a 70B model at 4-bit fit on one 80GB card once you reserve room for the KV cache at 8 concurrent users?

Deliverable: the three numbers, each with one sentence of interpretation. If 2c surprised you, re-read the notes section on the KV cache.

3. Base model vs. instruct model (45 min)

You have no base model to run, so do it as a written experiment:

- **3a.** Write three prompts where a base model's *completion* behaviour is the desired behaviour (e.g. continuing a list, mimicking a style, filling in a template).
- **3b.** Write the same three prompts as an instruct model would need them.
- **3c.** Write one prompt where an instruct model's *training* (refusals, helpfulness) gets in the way of a legitimate completion task, and how you would work around it.

Deliverable: seven short prompts with a one-line rationale each.

4. The decision: prompt, RAG, fine-tune, or continue pretraining (2h)

For each scenario below, choose **one** intervention and defend it in ≤ 5 sentences using cost, freshness, and failure modes. The trap in at least two of them is picking fine-tuning when it cannot work:

1. Customer-support answers over 40,000 tickets that rotate monthly; tone must match the brand.
2. Output must be strict JSON matching a schema, and the model keeps adding prose around it.
3. Domain jargon (semiconductor fab vocabulary) is misused; you have 2B tokens of in-domain text.
4. "Make it sound more like our CEO."

Checks: - In #1 you chose retrieval, not fine-tuning (knowledge lives in the tickets, and it rotates). - In #4 you chose few-shot prompting or an SFT on style — and said why a full fine-tune is overkill.

5. Failure-mode mitigation plan (1.5h)

Take the support bot from 2a. Produce a one-page table: failure mode $\rightarrow$ how you'd detect it $\rightarrow$ how you'd mitigate it $\rightarrow$ how you'd *measure* the mitigation worked. Cover at minimum: hallucinated policy answers, sycophancy ("are you sure?" flips the answer), context rot as the ticket thread grows, and prompt injection via pasted customer emails.

Check: the "measure" column references concrete eval sets or online metrics, not vibes. This table is your entrance exam for Module 15 [15-ai-evals].

6. Stretch: speculative decoding on paper (2h)

A 70B target model and a 1B draft model. Draft proposes 4 tokens; the target verifies in one forward pass. Assume the draft alone is accepted with probability p at each position.

- **6a.** Derive the expected number of target forward passes per generated token as a function of p .
- **6b.** At what p does speculative decoding break even with plain decoding of the 70B? State your assumption about the 1B model's relative speed.
- **6c.** Name two real-world properties of text that make p high (and one that makes it low).

Deliverable: the formula, the break-even estimate, and the three properties.